

NiceGUI-Pack 详细解析

`nicegui-pack` 是 NiceGUI 官方提供的**打包工具**，核心作用是将 NiceGUI 应用从“源码运行”转换为“可执行文件（EXE/APP/Deb 等）”，支持 Windows、macOS、Linux 全平台，无需用户安装 Python 环境即可运行，解决了 NiceGUI 应用分发的核心痛点。以下从工具定位、安装、使用方式、配置、注意事项等维度全面拆解。

一、核心定位与设计初衷

NiceGUI 应用基于 Python 运行，直接分发源码需用户安装 Python 及依赖（如 `nicegui`、`fastapi`、`uvicorn` 等），门槛较高。`nicegui-pack` 底层基于 `PyInstaller`（Python 主流打包工具），但针对 NiceGUI 的特性做了深度适配：

- 自动处理 NiceGUI 的静态资源（如 Vue 组件、CSS/JS 文件）；
- 简化打包配置，无需手动编写 PyInstaller 规范文件（`.spec`）；
- 内置对 NiceGUI 后台任务、WebSocket 等核心功能的兼容；
- 支持一键打包为单文件 / 文件夹形式，适配不同分发需求。

二、安装方式

`nicegui-pack` 需独立安装（需 Python 3.8+ 环境），推荐使用 `pip` 安装：

```
# 基础安装
pip install nicegui-pack

# 安装最新开发版（可选）
pip install git+https://github.com/zauberzeug/nicegui-pack.git
```

三、基础使用方式

1. 核心命令格式

```
nicegui-pack [OPTIONS] YOUR_APP_SCRIPT.py
```

核心参数说明：

参数	作用	示例
<code>-o/--output</code>	指定打包后文件的输出目录	<code>nicegui-pack -o dist my_app.py</code>
<code>--onefile</code>	打包为单个可执行文件（默认是文件夹形式）	<code>nicegui-pack --onefile my_app.py</code>
<code>--name</code>	指定可执行文件的名称	<code>nicegui-pack --name MyApp my_app.py</code>
<code>--windowed</code>	无控制台窗口（仅 Windows/macOS，适合 GUI 应用）	<code>nicegui-pack --windowed my_app.py</code>

参数	作用	示例
<code>--port</code>	指定应用运行的端口（默认随机端口）	<code>nicegui-pack --port 8080 my_app.py</code>

2. 最简示例

假设你的 NiceGUI 应用文件 `my_app.py` 内容如下：

```
from nicegui import ui

ui.label("Hello NiceGUI!")
ui.button("Click me", on_click=lambda: ui.notify("Clicked!"))

ui.run()
```

执行打包命令：

```
# 打包为文件夹形式（推荐，启动更快）
nicegui-pack my_app.py

# 或打包为单文件
nicegui-pack --onefile my_app.py
```

3. 运行打包后的程序

- **Windows**：进入输出目录（默认 `dist`），双击 `my_app.exe` 即可运行；
- **macOS**：进入 `dist` 目录，执行 `./my_app`（或右键打开 `my_app.app`）；
- **Linux**：进入 `dist` 目录，执行 `./my_app`。

运行后会自动启动 NiceGUI 应用，并在浏览器中打开（与源码运行一致）。

四、进阶配置

1. 自定义应用图标

支持为可执行文件添加图标（需 `.ico`（Windows）、`.icns`（macOS）、`.png`（Linux）格式）：

```
# windows
nicegui-pack --icon app.ico my_app.py

# macOS
nicegui-pack --icon app.icns my_app.py

# Linux
nicegui-pack --icon app.png my_app.py
```

2. 包含额外文件 / 目录

若应用依赖外部文件（如配置文件、图片、数据库），需通过 `--add-data`（Linux/macOS）或 `--add-binary`（Windows）指定，格式为 `源路径;目标路径`（Windows） / `源路径:目标路径`（Linux/macOS）：

```
# windows: 包含 config.json 和 images 目录
nicegui-pack --add-data "config.json;" --add-data "images;images" my_app.py

# Linux/macOS
nicegui-pack --add-data "config.json:" --add-data "images:images" my_app.py
```

3. 禁用自动浏览器打开

默认打包后运行会自动打开浏览器，若需禁用，可在代码中设置 `ui.run(show=False)`，或打包时指定 `--no-open`：

```
nicegui-pack --no-open my_app.py
```

4. 自定义端口与主机

打包时指定固定端口和主机（避免随机端口）：

```
nicegui-pack --host 0.0.0.0 --port 8000 my_app.py
```

五、关键注意事项

1. 打包后程序的运行路径问题

若应用中读取 / 写入本地文件（如 `open("data.txt", "r")`），需注意：

- 单文件打包（`--onefile`）运行时，会先解压到临时目录（如 Windows 的 `%TEMP%`），相对路径会指向临时目录；
- 推荐使用 `pathlib` 获取程序实际运行路径：

```
from pathlib import Path
import sys

# 获取程序所在目录
if getattr(sys, 'frozen', False):
    # 打包后运行
    app_dir = Path(sys.executable).parent
else:
    # 源码运行
    app_dir = Path(__file__).parent

# 读取文件
with open(app_dir / "data.txt", "r") as f:
    data = f.read()
```

2. 依赖兼容性

- `nicegui-pack` 仅兼容 NiceGUI v1.0+ 版本，低版本可能出现打包失败；

- 若应用依赖特殊第三方库（如 `pytorch`、`opencv`），需确保库支持 PyInstaller 打包，必要时手动安装对应依赖的二进制包。

3. 单文件 vs 文件夹打包

打包形式	优点	缺点
单文件（ <code>--onefile</code> ）	分发方便（仅一个文件）	启动慢（每次运行需解压到临时目录）、临时目录占用磁盘
文件夹	启动快、可修改配置文件	分发文件多、体积较大

4. macOS/Linux 权限问题

- macOS 打包后可能提示“无法验证开发者”，需右键选择“打开”，或在“系统设置 - 隐私与安全性”中允许运行；
- Linux 需为可执行文件添加权限：`chmod +x dist/my_app`。

5. 后台任务的兼容

NiceGUI 的 `background_tasks` 无需额外配置，`nicegui-pack` 已自动适配，但需确保任务中无硬编码的绝对路径（改用相对路径 / 运行时路径）。

六、常见问题与解决方案

1. 打包失败：“Missing required keys”

原因：NiceGUI 版本过低，升级到最新版即可：

```
pip install --upgrade nicegui
```

2. 运行打包程序提示“找不到模块”

原因：第三方库未被 PyInstaller 识别，需手动指定 `--hidden-import`：

```
nicegui-pack --hidden-import=xxx my_app.py
```

3. 单文件打包后运行卡顿

解决方案：改用文件夹打包，或优化应用启动逻辑（如延迟加载非核心组件）。

4. macOS 打包后无法打开

原因：缺少签名，可通过以下命令绕过（仅测试用）：

```
xattr -cr dist/my_app.app
```

七、与手动使用 PyInstaller 的差异

特性	<code>nicegui-pack</code>	手动 PyInstaller
NiceGUI 静态资源处理	自动处理	需手动添加 <code>--add-data</code> 包含 NiceGUI 资源
配置复杂度	极简（仅需指定源码文件）	需编写 <code>.spec</code> 文件，配置繁琐

特性	<code>nicegui-pack</code>	手动 PyInstaller
后台任务 / WebSocket 兼容	内置适配	需手动配置钩子 (hook) 文件
端口 / 浏览器控制	内置参数支持	需修改源码或手动传参